Classes

JS and many other languages are Object Oriented meaning that everything is an object belonging to a class. This programming paradigm is called **OOP (Object Oriented Programming)**.

Think of a class as a blueprint, from which any number of objects can be made. The objects made from a class inherit all the properties and methods of the class.

To make a class, start with the keyword class, followed by the name of your class. Variable naming rules apply, although, by convention class names are capitalized. After the class name comes a pair of curly braces:

```
class Car {
}
```

The **constructor function** is the first thing defined inside the class curly braces. The constructor has all the properties of the class as its parameters. These will be assigned values when an object of the class is instantiated.

```
class Car {

    constructor(make, model, year, color, miles, mpg) {

    }

}
```

Inside the consructor, each **class property** is defined using the syntax `this.key`, so the syntax for the `model` property is `this.model`:

```
class Car {

    constructor(make, model, year, color, miles, mpg) {

        this.make = make;
        this.model = model;
        this.year = year;
        this.color = color;
        this.miles = miles;
        this.mpg = mpg;

    }

}
```

A recap of the `this` keyword:

In the **global scope**, `this` ris the Window, the top-level object.

```
    console.log(this); // Window
```

Inside a function, `this` is the object that has the event attached to it that calls the function. So, if a button click calls a function, `this` refers to the button itself--not the click, which is the `event`. In the function, this also goes by the designation `event.target`.

In an object, `this` *is* the object. So, `this.year` is the `year` property of the object.

Assign properties using `this.key` syntax. Set each equal to the parameter, the value of which is supplied when an object of the class is instantiated. All the properties go inside the constructor function.

Class Method

A **method** is an **object property** that has a function for its value. When the method is called on the object, the method returns a value.

Add two methods to our class: once called `describe` and another called `listForSale`. The latter returns a OR SALE listing, which uses all the other properties to make the listing.

When a method refers to a property of the same object, it uses the `this` keyword: `this.color`.

```
    class Car {

        constructor(make, model, year, color, miles, mpg) {

            this.make = make;
            this.model = model;
            this.year = year;
            this.color = color;
            this.miles = miles;
            this.mpg = mpg;

            this.describe = function() {
                return `${this.year} ${this.color} ${this.make}
${this.model}`;
            }

            this.listForSale = function() {
                return `FOR SALE: ${this.color} ${this.year} ${this.make}
${this.model}. Gets ${this.mpg} mpg. Only ${this.miles} miles`;
            }

        }
    }
```

instantiating an object from the class

When an object is made from a class, this is called **object instantiation**. The instantiation of the object causes the **constructor function** to run automatically.

Instantiate objects of the Car class using the constructor method. This uses the `new` keyword, which should be familiar from our work with the `Date` object.

When the object is instantiated, all the properties of the object get their values set by the arguments.

```
let car1 = new Car('Honda', 'Accord', 2010, 'red', 82345, 31);
let car2 = new Car('Dodge', 'Charger', 2020, 'black', 12345, 15);
```

Output properites of the objects:

```
console.log(car1.make);
console.log(car1.mpg);
console.log(car1.cylinders);

console.log(car2.model);
console.log(car2.miles);
console.log(car2.color);
```

Call the methods on each object.

```
console.log(car1.describe());
console.log(car2.describe());

console.log(car1.listForSale());
console.log(car2.listForSale());
```